

- [Entering a program](#)
 - [Simple sequential programs](#)
 - [Simple recursive programs](#)
 - [Interaction between processes, Concurrency](#)
 - [Master and Slaves, error handling](#)
 - [Robustness in Erlang, and use of a graphics package](#)
 - [Erlang using UNIX sockets](#)
 - [The use of open_port/1](#)
 - [Socket communication between Erlang and C](#)
 - [Implementing Talk with Distributed Erlang](#)
 - [Generating a parser for Datalog](#)
-

Entering a program

Type the **demo:double** example into a file called **demo.erl**. Use your favourite text editor.

Start Erlang.

Give the command **c:c(demo)**. to compile the file.

Try running the query:

```
demo:double(12) .
```

This is just to test if you can get the system started and can use the editor together with the Erlang system.

[Back to top](#)

Simple sequential programs

1. Write functions **temp:f2c(F)** and **temp:c2f(C)** which convert between centigrade and Fahrenheit scales.
(hint $5(F-32) = 9C$)

2. Write a function **temp:convert(Temperature)** which combines the functionality of **f2c** and **c2f**. Example:

```
> temp:convert({c,100}) .  
=> {f,212}  
> temp:convert({f,32}) .  
=> {c,0}
```

3. Write a function **mathStuff:perimeter(Form)** which computes the perimeter of different forms. **Form** can be one of:

```
{square,Side}  
{circle,Radius}  
{triangle,A,B,C}
```

[Back to top](#)

Simple recursive programs

1. Write a function **lists1:min(L)** which returns the minimum element of the list **L**.
2. Write a function **lists1:max(L)** which returns the maximum element of the list **L**.
3. Write a function **lists1:min_max(L)** which returns a tuple containing the min and max of the list **L**.

```
> lists1:min_max([4,1,7,3,9,10])  
{1, 10}
```

4. Write the function **time:swedish_date()** which returns a string containing the date in swedish YYMMDD format:

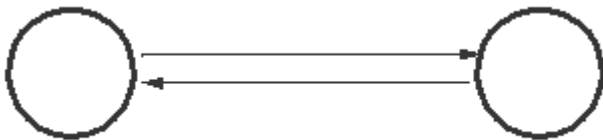
```
> time:swedish_date()  
"080901"
```

*Hints: trying looking up **date()** and **time()** in the manual, You may also need **integer_to_list/1**. Try giving the shell queries to see what these BIF's do.)*

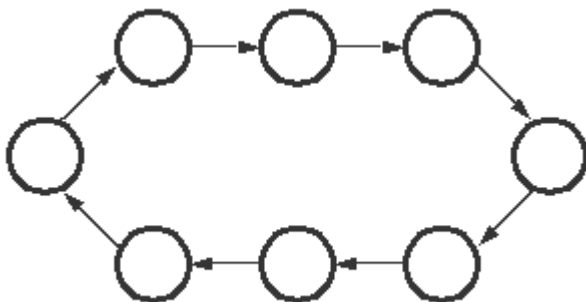
[Back to top](#)

Interaction between processes, Concurrency

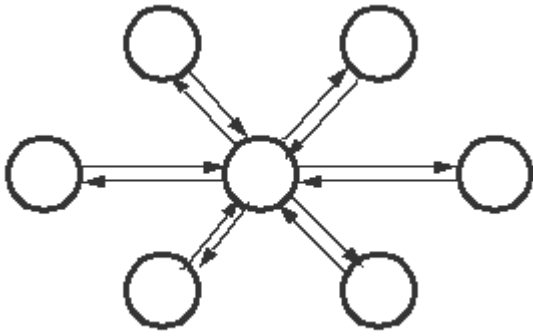
1. Write a function which starts 2 processes, and sends a message M times forwards and backwards between them. After the messages have been sent the processes should terminate gracefully.



- 2) Write a function which starts N processes in a ring, and sends a message M times around all the processes in the ring. After the messages have been sent the processes should terminate gracefully.



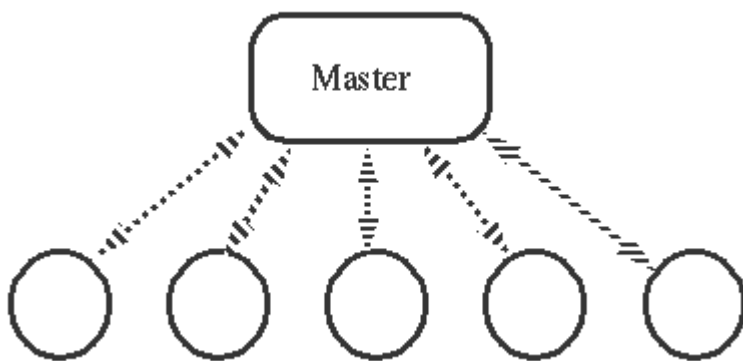
- 3) Write a function which starts N processes in a star, and sends a message to each of them M times. After the messages have been sent the processes should terminate gracefully.



[Back to top](#)

Master and Slaves, error handling

This problem illustrates a situation where we have a process (the master) which supervises other processes (the slaves). In a real example the slave could, for example, be controlling different hardware units. The master's job is to ensure that all the slave processes are alive. If a slave crashes (maybe because of a software fault), the master is to recreate the failed slave.



Write a module **ms** with the following interface:

start(N) - Start the master and tell it to start **N** slave processes. Register the master as the registered process **master**.

to_slave(Message, N) - Send a message to the master and tell it to relay the message to slave **N**. The slave should exit (and be restarted by the master) if the message is **die**.

The master should detect the fact that a slave process died and restart it and print a message that it has done so.

The slave should print all messages it receives except the message **die**

Hints:

The master should trap exit messages and create links to all the slave processes.

The master should keep a list of the process id's (pid's) of the slave processes and their associated numbers.

Example:

```
> ms:start(4).
=> true
> ms:to_slave(hello, 2).
```

```
=> {hello,2}
Slave 2 got message hello
> ms:to_slave(die, 3).
=> {die,3}
master restarting dead slave3
```

[Back to top](#)

Robustness in Erlang, and use of a graphics package

A robust system makes it possible to survive partial failure, i.e if some parts of the system crashes, it should be possible to recover instead of having a total system crash. In this exercise we will build a tree-like hierarchy of processes that can recover if any of the tree-branches should crash.

To illustrate this we are going to use the Interviews interface and having each process represented by a window on the screen.

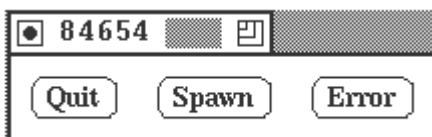
Exercise: Create a window containing three buttons: **Quit** , **Spawn** , **Error**.

The **Spawn** button shall create a child process which displays an identical window.

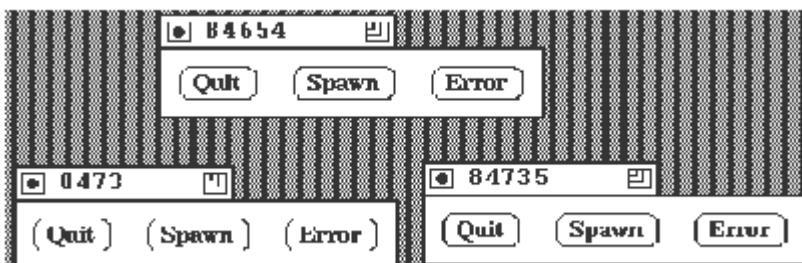
The **Quit** button should kill the window and its child windows.

The **Error** button should cause a runtime error that kills the window (and its children), this window shall then be restarted by its parent.

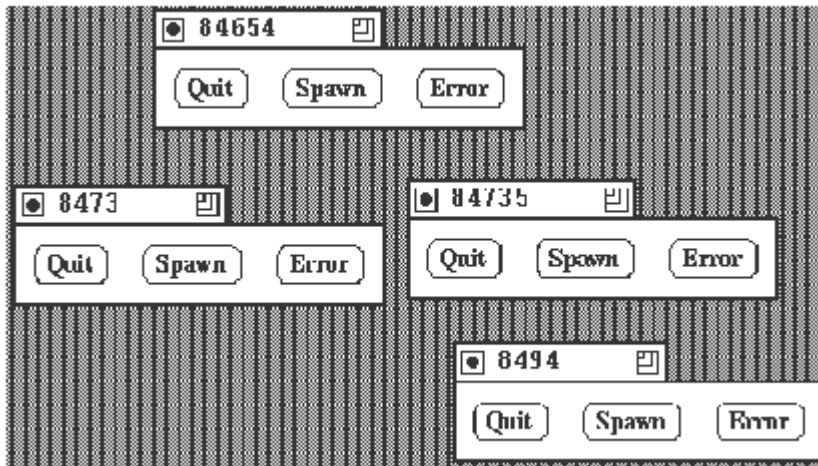
Example: When we start our program, a window like this should appear:



Let us now press the **Spawn** button twice and as result, two child windows will pop up on our screen. Our screen will now look something like this:



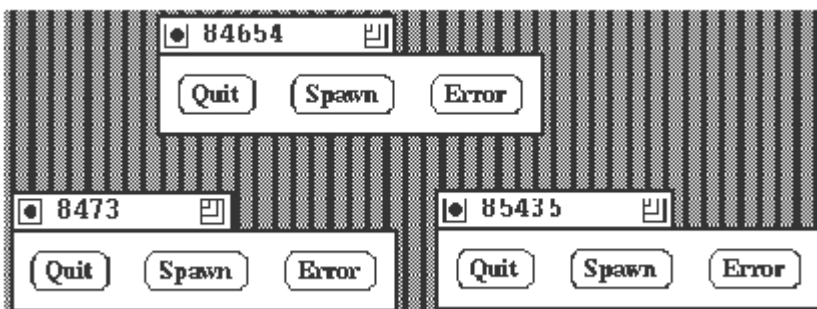
As you can see the windows are tagged with a number so it is easier for us to refer to them. The parent window has the number 84654 and its two childs number 8473 and 84735. Each child may have childs of its own, e.g press the **Spawn** button on window 84735 and we will have the following picture:



The window 8494 is a child to window 84735.

Now let's press the **Error** button in window 84735. Now a runtime error will occur in the process for that window and the process (and the window) will die. This shall cause the child 8494 also to die and the parent 84654 to start up a new child window.

The result will look something like this:



The new child got number 85435.

[Back to top](#)

Erlang using UNIX sockets

Do you want to talk with a friend on another machine? Shouldn't it be nice to have a shell connected to your friend and transfer messages in between?

This can be implemented using the client/server concept with a process on each side listening to a socket for messages.

- Write a distributed (client/server) message passing system. The system shall be built upon the Erlang interface to the BSD unix sockets.
- The server host name will be given as input argument in order to start the client.
- A prompt shall be displayed both on the server and the client side. The user shall give a string followed by a RETURN as a message. The message will be transferred and displayed to the user on the other side.
- An empty input string (on either side) will end the session.

(Hints: read the man-page for the socket interface, also in order to read the command line, use `io:get_line/1`)

[Back to top](#)

The use of open_port/1

Use the `open_port({spawn,Name})` BIF in order to start an external UNIX process.

- The Erlang program shall open a port, sending some data through the port, have the data echoed back and then printout the received data.
- The C program that will run in the UNIX process shall consist of an eternal loop starting with a read from file descriptor 0 (stdin) and end with a write on file descriptor 1 (stdout) before it iterates again.

Note: The first two bytes read in the C program contains the length in bytes of the data to follow (*Make sure that you are reading as much as the length indicates*).

(Hints: read the User's Guide p.37 and the BIF Guide p.11)

[Back to top](#)

Socket communication between Erlang and C

Write Erlang and C programs which do the following:

- The Erlang program shall create a socket, waiting for accept, sending some data through the socket, have the data echoed back, printout the received data, and then close the socket.
- The C program that will run in the UNIX process shall take the hostname of the host you are going to communicate with as a parameter. Set up a socket to that host and then echo data as in the previous exercise.

(Hints: Include the files listed below. The main routine will look much the same as in the previous example. The code to setup the socket is found on the next page)

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <errno.h>

/*--- Setup a socket to Host using Port , return the filedesc. ---*/

static int setup_socket( hostname , port )
    char *hostname;
    int port;
{
    struct sockaddr_in serv_addr;
    int sockfd;
    struct hostent *hp;

    /*--- Get the address of the host ---*/

    if ((hp = gethostbyname( hostname )) == (struct hostent*) 0) {
        perror("From gethostbyname \n");
        exit(-1);
    }

    /*--- Fill in the address to the remote system ---*/

    bzero((char *) &serv_addr , sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;          /* Protocol family */
    serv_addr.sin_port   = htons( port );    /* The port number */
}
```

```

bcopy((hp->h_addr_list[0] , /* The net address to the host */
      (char *) &serv_addr.sin_addr.s_addr ,
      hp->h_length);

/*--- Create the socket ---*/

if ( (sockfd = socket( AF_INET , SOCK_STREAM , 0 )) < 0 ) {
    perror("setup_socket: socket");
    exit(-1);
}

/*--- Connect to the other system ---*/

if (connect(sockfd, &serv_addr, sizeof(serv_addr)) < 0) {
    perror("setup_socket: connect");
    exit(-1);
}
else
    return sockfd;
}; /* setup_socket */

```

[Back to top](#)

Implementing Talk with Distributed Erlang

Make a simple Talk program that makes it possible to chat with friends at other nodes/hosts.

- First you and your friend must create identical **.erlang.cookie** files, e.g:

```
echo -n "dh32d8yhd8" > ~/.erlang.cookie
```

- Make sure nobody else can read it:

```
chmod 400 ~/.erlang/cookie ).
```

- Then start a distributed Erlang node:

```
erl -name bill -cookie
```

- Now start your program. It should begin with prompting for the other node name, and then prompting for messages that will be sent to your friend for each carriage-return.

Hints: Your program should consist of two registered processes one for reading from the terminal and the other one for receiving messages from the other node then writing them to the terminal.

[Back to top](#)

Generating a parser for Datalog

By using the Yecc parser generator we will generate a parser that will accept Datalog programs according to the specified grammar below. We will also have to use/modify a scanner (lexical analyser) to suit our purpose.

The syntax for Datalog can be described by the following grammar:

```

PGM -> e
PGM -> CLAUSE PGM
CLAUSE -> LITERAL TAIL .

```

```
LITERAL -> predsymb PARLIST
PARLIST -> e
PARLIST -> ( ARGLIST )
ARGLIST -> TERM ARGTAIL
ARGTAIL -> e
ARGTAIL -> , ARGLIST
TERM -> csym
TERM -> varsym
TAIL -> e
TAIL -> :- LITLIST
LITLIST -> LITERAL LITTAIL
LITTAIL -> e
LITTAIL -> , LITLIST
```

This grammar will accept Datalog programs, for example:

```
path(stockholm, uppsala).
```

or:

```
route(X,Y) :- path(X,Z), path(Z,Y).
```

The tokens produced by the scanner are defined as:

```
predsym = lc(lc + uc + digit)* | digit*
csym = lc(lc + uc + digit)* | digit*
varsym = uc(lc + uc + digit)*
lc = any lowercase letter
uc = any uppercase letter
digit = any digit
```

To be able to solve this exercise you will have to read the man-page for Yecc (**erl -man yecc**). Good Luck !!

[Back to top](#)